Graph-Based Recommendations with Personalized PageRank

Implementation evidence and a deployment decision for a MovieLens 100K recommender

Ilya Bochkarev · LLM4Rec · HSE University · HW6 · 22 October 2026
Repository: github.com/bochkarev87/homework_6

1. Executive summary

I built the required user–movie graph from positive training interactions, implemented global PageRank and seeded Personalized PageRank (PPR) with TensorFlow.js, and evaluated popularity, global PageRank, PPR and matrix factorization (MF) on one shared test cohort. All seven assignment stubs are implemented, all 12 self-checks pass, and every reported propagation run converged below the L1 tolerance of `1e-6`.

0.1122	0.0760	0.0312
PPR RECALL@10, TEST (861 USERS)	POPULARITY RECALL@10	MF RECALL@10 (CHECKPOINT 4)

> **Deployment recommendation: use both, in two different sections.** Keep MF for a predicted-rating shelf and use PPR for the main graph-based discovery row. PPR retrieved held-out relevant movies more than three times as often as MF (Recall@10 11.22% vs 3.12%) and 47.6% more often than popularity, while MF produced much less popular lists (mean training popularity 94.3 vs 244.2) and only 4 of 10 titles overlapped with PPR for the two demonstration users. These are complementary offline signals, not proof that two rows will improve engagement. A user-level online experiment is required before rollout.

1.1 Requirement coverage

Assignment requirement	Where it is addressed
User–movie graph from training ratings `>= 4.0`; missing ratings unobserved	Section 3; `graph.js`
Global PageRank and user-seeded PPR with TensorFlow.js; alpha is the link-following probability	Section 4.1; `graph.js`
Transition normalization, probability conservation, convergence and zero-degree handling	Section 4.3; `verification.json`
Alpha selected by validation Recall@10, never by test outcomes	Section 5.1; `results.csv`
Popularity, global PageRank, PPR and MF compared on the same eligible users and candidate movies	Section 5; 861-user shared cohort
Validation excludes training ratings; test excludes training and validation ratings	Section 3; Section 5
Test Top-10 lists for the same two users with titles, genres and scores	Sections 6.2–6.3; `recommendations.csv`
One-factor-at-a-time demo: users A/B at alpha 0.85, then user A at 0.85 versus 0.50	Section 6
Report test Recall@10, exclusions and training popularity; PPR scores are not ratings or purchase probabilities	Section 5; Section 6.2; Section 8

2. Related work

This project combines link analysis, personalized search and latent-factor collaborative filtering. Brin and Page introduced PageRank as a global authority score for web pages (Brin and Page 1998). Our global PageRank baseline applies the same fixed-point update to a bipartite user–movie graph. Because a walk alternates between user and movie nodes, the global stationary mass is driven by degree and connectivity rather than by one user's history; it is therefore best read as a graph-smoothed popularity signal, not as a personalized recommender.

Jeh and Widom introduced personalized PageRank, which replaces the uniform restart with a distribution concentrated on a seed (Jeh and Widom 2003). Our PPR uses the same random-walk with restart, with a one-hot restart at the target user. We compute it exactly by batched power iteration on the 2,554-node graph rather than by precomputed basis vectors or approximate top-k propagation. The algorithmic contribution here is therefore an implementation and evaluation study, not a new PPR variant: the batched path makes 861 user-specific runs practical in the browser, and its output is checked against the single-seed implementation.

Matrix factorization represents users and movies by latent factors learned from observed ratings (Koren, Bell and Volinsky 2009). Our MF follows the week-5 explicit-rating model: a 16-dimensional dot product plus the global training mean, trained with RMSE loss and Adam. This differs from pairwise ranking objectives such as BPR, which optimize the order of positive and negative items directly (Rendle et al. 2009). That objective mismatch is one reason to expect weaker Recall@10 from an RMSE-trained model, and it is why the report treats MF as a predicted-rating shelf rather than as a like-for-like ranking competitor.

MovieLens 100K is the standard benchmark used here (Harper and Konstan 2015). Published MovieLens results are not directly comparable to our numbers because they use different splits, candidate sets and ranking protocols. We therefore compare only methods trained and evaluated inside this repository on the same 861-user cohort, and we report the exact exclusion rules rather than matching a published leaderboard.

Positioning. The assignment is not a novel recommendation algorithm. Its contribution is a reproducible browser harness that compares four ranking signals under one cohort, an exact batched PPR implementation validated against a single-seed reference, and a disclosed correction to the starter MF regularizer.

2.1 Ideas from the sources that shaped my solution

I did not use the sources only as background citations. Each one supplied a specific idea that helped me make a design decision or interpret a result.

- **Brin and Page (1998).** The useful idea was that node importance can be defined recursively by the importance of nodes that point to it. I used that idea to build the global PageRank baseline, but I kept it as a graph-smoothed popularity signal rather than treating it as personalized recommendation.
- **Jeh and Widom (2003).** The idea that helped most was replacing the uniform restart in PageRank with a restart distribution concentrated on one seed. That is exactly the problem in this assignment: global PageRank is not user-specific. I implemented PPR with a one-hot restart at the target user, then batched the same propagation over the evaluation cohort.
- **Koren, Bell and Volinsky (2009).** The latent-factor idea explained why MF produces a predicted-rating shelf rather than a graph-proximity ranking. I reused the learned user and movie factors and the global training mean, and I kept the MF score on its own scale instead of comparing it numerically with PPR probabilities.
- **Rendle et al. (2009).** The BPR idea of optimizing a pairwise ordering helped me interpret the MF gap: an RMSE-trained model is optimized for rating prediction, not for retrieving relevant movies into a Top-10. I therefore treated MF as a rating-oriented baseline, not as a like-for-like ranking competitor.
- **Harper and Konstan (2015).** The dataset description clarified that MovieLens 100K contains explicit ratings and rating timestamps rather than viewing sessions. That shaped the warm-user evaluation, the

chronological split and the caution that a missing rating is unobserved data, not a zero preference.

The combination is the core of my method: PageRank supplies the propagation rule, personalized restart supplies user conditioning, MF supplies a rating-oriented reference point, BPR explains the ranking-objective mismatch, and the MovieLens description defines what the data can and cannot support.

3. Data, graph and evaluation population

The application loads the same MovieLens 100K files and the same per-user chronological 80/10/10 split as week 5. Missing ratings remain unobserved; they are never treated as zero preferences. Only training ratings `>= 4.0` create graph edges.

Quantity	Value	Note
Ratings loaded	100,000	MovieLens 100K
Movies loaded	1,682	`u.item`
Train / validation / test records	79,619 / 9,596 / 10,785	Chronological within user
Rating-matrix density	6.30%	100,000 / (943 × 1,682)
User nodes / movie nodes	943 / 1,611	Observed in the training split
Graph nodes / positive edges	2,554 / 45,602	No padding node
Zero-degree users / movies	1 / 253	Policy below
Training-movie candidate pool	1,611	All training movies
Shared TEST cohort	861	943 candidate users − 82 excluded
Validation cohort	846	Used only for alpha selection

Thirteen relevant test movie IDs are absent from the training candidate pool: 913, 1106, 1130, 1458, 1498, 1525, 1613, 1625, 1645, 1646, 1650, 1651, 1674. Eighty-two users are excluded from the shared cohort because they have no eligible relevant test movie after filtering (one user also has no positive training edge). The same 861 users are used for all four methods; the alpha sweep uses the 846-user validation cohort and does not inspect test outcomes.

The split is chronological within each user rather than globally time-consistent, and MovieLens timestamps are rating times rather than viewing sessions. This is a warm-user evaluation; it says nothing about cold start.

4. Implementation and verification

4.1 Graph and propagation

Users occupy node indices `0..nUsers-1` and movies follow, so no padding node enters the graph or the candidate pool. For each positively rated training pair, the user row is normalized over that user's positive movies and the movie row over the users who rated it positively. The propagation uses the two off-diagonal blocks R and R^T rather than materializing a dense 2,554 × 2,554 matrix.

Dangling mass is not discarded: zero-degree rows contribute their mass to the restart distribution v, using the same rule in global PageRank and PPR. A run stops when the L1 difference between consecutive distributions is below `1e-6`, or at 500 iterations; the iteration cap is never treated as proof of convergence.

PPR for evaluation is computed once per `(graph, alpha, split)` for the whole cohort in a single batched TensorFlow.js propagation, then reused for each user's Top-10. This keeps the main propagation in tensor operations and avoids repeating a per-user walk 861 times. The batched path was checked against the single-

seed reference: at alpha 0.85 the two ran 94 and 91 iterations and differed by at most `3.96e-9` in movie scores, which is float32 noise.

4.2 MF reference

MF keeps the week-5 scoring definition: dot product of 16 learned factors plus the global training mean, used as a predicted rating. It is not converted into a probability and its scores are never compared numerically with PPR walk probabilities. Training uses Adam, learning rate 0.01, regularization 0.05, batch size 512, 30 epochs and a fixed seed; the best validation-RMSE checkpoint is epoch 4 with validation RMSE 0.9779.

Disclosed correction. The starter's L2 term summed over every factor row, which collapsed all embeddings toward zero in the first epoch (validation RMSE stuck at 1.1830, checkpoint 1). I replaced it with the week-5 per-batch mean objective, `(sum(u²) + sum(i²)) × reg / batchSize`. This changes regularization only; the scoring rule and the validation-checkpoint rule are unchanged.

4.3 Verification evidence

Check group	Result	Evidence
Reference graph, uniform restart, alpha 0.85	PASS	Ranking C > A > B > D, matches the provided fixture
Reference graph, restart at A, alpha 0.85	PASS	Ranking A > C > B > D; D stays zero
Row normalization	PASS	Six positive-degree rows sum to 1; two dangling rows are zero
Restart distributions	PASS	Uniform and one-hot are nonnegative and sum to 1
Probability conservation	PASS	Total 1.00000027 after updates
Nonnegative stationary scores	PASS	All synthetic scores nonnegative
One extra update stability	PASS	L1 = 7.97e-7
Dangling-node fixture	PASS	Mass 1.00000028 / 1.00000000
Matrix orientation	PASS	Probability collects at the pointed-to node
Top-10 filtering and tie order	PASS	Previously rated items removed; ties by ascending movie ID
Graph mapping	PASS	2,554 = 943 users + 1,611 movies
Public API surface	PASS	Ten functions plus `ALPHA_GRID` and `MF_INFO`

Overall: 12 passed, 0 failed, 0 pending. The batch-vs-single PPR cross-check is recorded as an additional PASS in `verification.json`. Recorded single-run timings on this device: graph build 11 ms, global PageRank 70 ms, one PPR 70 ms, validation sweep 2.0 s, four-method comparison 7.9 s, end-to-end 11.4 s. These are evidence from one WebGL run and are not compared across devices.

4.4 Hand-computed propagation check

To check the code against arithmetic rather than only against itself, take the four-node reference graph with p_0 `= [0.25, 0.25, 0.25, 0.25]` and `alpha = 0.85`. Before restart, A receives C's mass `0.25`; B receives half of A's mass, `0.125`; C receives half of A plus B plus D, `0.125 + 0.25 + 0.25 = 0.625`; D receives nothing. Adding the uniform restart gives p_1 `= 0.85 × [0.25, 0.125, 0.625, 0] + 0.15 × 0.25`, i.e. `[0.25, 0.14375, 0.56875, 0.0375]`.

With the application loaded, this DevTools console call forces one update:

```
Array.from(pageRank(0.85, {
  transition: REFERENCE_GRAPH.transition,
  maxIter: 1
}).p)
// [0.25, 0.14375001192092896, 0.5687500238418579, 0.03750000149011612]
```

The code returns the hand-computed vector to float32 precision; the largest coordinate error is `2.4e-8`. Running to convergence returns `[0.37252694, 0.19582376, 0.39414933, 0.03750000]` in 28 iterations with residual `9.54e-7`, within `1.6e-7` of the supplied reference vector and below the `1e-6` tolerance. This is the requested hand-computed check: it verifies one propagation step, not the full 861-user evaluation. The fixed point is reported by the code but was not re-derived by hand.

5. Retrieval results

Recall@10 counts held-out test movies rated `>= 4.0` that appear in the training candidate pool and pass the test exclusion rule (training and validation ratings removed). A relevant movie is retrieved if it appears in the user's Top-10; the reported value is the macro average over the 861 shared users.

Method	Alpha	Recall@10	Mean list popularity	Iterations	Residual
Popularity	—	0.0760	272.0	—	—
Global PageRank	0.50	0.0797	257.8	19	5.47e-7
PPR	0.50	**0.1122**	244.2	22	5.01e-7
MF, checkpoint 4	—	0.0312	94.3	—	—

PPR is 47.6% above popularity and 259.0% above MF in macro Recall@10 on the same users. Global PageRank is only 0.4 percentage points above popularity. That is consistent with a mostly global popularity signal; it was not independently tuned. The same selected alpha (0.50) is used for global PageRank and PPR, as required, so this is not evidence that PageRank was optimized.

5.1 Alpha selection on validation only

Alpha	Validation Recall@10	Validation cohort
0.50	**0.1145**	846
0.70	0.1048	846
0.85	0.0900	846
0.95	0.0738	846

The validation sweep favors the lowest tested alpha, 0.50; larger alpha values did not improve Recall@10 here. Test outcomes were not used to choose alpha. MF and PPR use different validation objectives (RMSE vs Recall@10), so their test recalls are descriptive rather than a controlled test of one algorithm's superiority.

6. Two-user one-factor-at-a-time demonstration

Users 1 and 2 stay fixed. Only one factor changes per comparison: seed, or alpha, never both.

Setting	User	Alpha	Mean list popularity	Shared with baseline
Baseline	1	0.85	245.5	10/10
Change seed only	2	0.85	254.9	0/10
Change alpha only	1	0.50	228.4	7/10

Changing the restart user replaced the whole list; changing alpha from 0.85 to 0.50 retained seven titles. Neither result says which setting is “better” in business terms; it shows that the restart seed and the link-following probability act on different parts of the ranking. The two users' lists also overlap differently by method: 1/10 for popularity, 1/10 for global PageRank, 1/10 for PPR and 4/10 for MF.

6.1 Training histories

User	Train / val / test	Positive train	Top positive-training genres
1	217 / 27 / 28	131	Drama 61, Comedy 39, Action 36, Sci-Fi 27, Thriller 25
2	49 / 6 / 7	32	Drama 18, Romance 11, Comedy 11, Action 5, Crime 5

User 1's positive history includes *Monty Python and the Holy Grail*, *The Empire Strikes Back*, *Jean de Florette*, *Reservoir Dogs* and *Dead Poets Society*. User 2's includes *The English Patient*, *In & Out*, *The Full Monty*, *Rosewood* and *Hoodlum*.

6.2 PPR Top-10 at the selected alpha (0.50)

Rank	User 1: movie	Genres	Pop.	User 2: movie	Genres	Pop.
1	Fargo (1996)	Crime; Drama; Thriller	377	Return of the Jedi (1983)	Action; Adventure; Romance; Sci-Fi; War	339
2	Contact (1997)	Drama; Sci-Fi	307	Dead Man Walking (1995)	Drama	182
3	The English Patient (1996)	Drama; Romance; War	279	Twelve Monkeys (1995)	Drama; Sci-Fi	234
4	Schindler's List (1993)	Drama; War	224	The Rock (1996)	Action; Adventure; Thriller	215
5	One Flew Over the Cuckoo's Nest (1975)	Drama	202	The Silence of the Lambs (1991)	Drama; Thriller	308
6	Casablanca (1942)	Drama; Romance; War	197	Independence Day (1996)	Action; Sci-Fi; War	207
7	Star Trek: First Contact (1996)	Action; Adventure; Sci-Fi	189	Raiders of the Lost Ark (1981)	Action; Adventure	312
8	Trainspotting (1996)	Drama	164	Chasing Amy (1997)	Drama; Romance	146
9	Sense and Sensibility (1995)	Drama; Romance	178	Trainspotting (1996)	Drama	164
10	E.T. the Extra-Terrestrial (1982)	Children's; Drama; Fantasy; Sci-Fi	167	Mr. Holland's Opus (1995)	Drama	160

PPR scores for these lists are long-run walk probabilities (user 1 rank 1: 0.000686; user 2 rank 1: 0.000697), not ratings or purchase probabilities. Full lists with scores and training popularity are in `recommendations.csv`.

6.3 MF Top-10 (checkpoint 4)

Rank	User 1: movie	Genres	Pop.	User 2: movie	Genres	Pop.
1	The Wrong Trousers (1993)	Animation; Comedy	90	Persuasion (1995)	Romance	28
2	A Close Shave (1995)	Animation; Comedy; Thriller	85	Casablanca (1942)	Drama; Romance; War	197
3	Casablanca (1942)	Drama; Romance; War	197	M*A*S*H (1970)	Comedy; War	131
4	Lawrence of Arabia (1962)	Adventure; War	134	Ran (1985)	Drama; War	51
5	Pather Panchali (1955)	Drama	7	Three Colors: Red (1994)	Drama	56
6	The Maltese Falcon (1941)	Film-Noir; Mystery	91	Pather Panchali (1955)	Drama	7
7	When We Were Kings (1996)	Documentary	29	The Third Man (1949)	Mystery; Thriller	63
8	Rear Window (1954)	Mystery; Thriller	152	Lawrence of Arabia (1962)	Adventure; War	134
9	Aparajito (1956)	Drama	4	The Quiet Man (1952)	Comedy; Romance	41
10	One Flew Over the Cuckoo's Nest (1975)	Drama	202	The Maltese Falcon (1941)	Film-Noir; Mystery	91

MF scores are predicted ratings on a different scale (for example 5.24 for user 1's first item) and are not comparable to PPR probabilities.

7. Business and algorithmic analysis

Should the platform keep MF, replace it with PPR, or use both?

Use both, in different sections. PPR should power the primary discovery row because it has the strongest retrieval evidence: Recall@10 0.1122 versus 0.0797 for global PageRank, 0.0760 for popularity and 0.0312 for MF on the shared 861-user cohort. It is also genuinely user-conditioned: changing only the restart user replaced all ten baseline titles, while changing only alpha retained seven. MF should remain as a predicted-rating shelf, not as the main discovery row: its retrieval is weaker, but its lists are much less popularity-driven (mean training popularity 94.3 versus 244.2 for PPR) and only 4/10 titles overlap with PPR for the two users.

The lists make the distinction concrete. User 1's PPR row starts with *Fargo* (1996) (Crime; Drama; Thriller), while the MF shelf starts with *The Wrong Trousers* (1993) (Animation; Comedy). User 2's PPR row starts with *Return of the Jedi* (1983) (Action; Adventure; Romance; Sci-Fi; War), while the MF shelf starts with *Persuasion* (1995) (Romance). These are different recommendation intents, not two rankings of one score.

The proposed sections have distinct purposes. The PPR row is a graph-based discovery surface: “users with your positive history also connect to these movies.” The MF row is a rating-style shelf: “movies the fitted model predicts you would rate highly.” Showing both gives the platform a recall-oriented list and a rating-oriented list without pretending that one score scale is the other. This is a hypothesis supported by offline retrieval and list overlap, not a demonstrated engagement gain.

Online experiment before rollout

Run a user-level randomized controlled trial for two weeks with three arms: (A) PPR discovery row only, (B) MF predicted-rating row only, (C) both rows in their proposed sections. Assign each user by a stable hash of user ID to one arm; keep assignment fixed for the experiment so a user does not cross arms, and analyze by intention to treat. This avoids contamination between recommender variants.

- **Primary outcome:** 7-day qualified watch-through rate on the recommendation surface (or click-through rate if watch completion is not observable), measured per assigned user, not per impression.
- **Guardrails:** catalog coverage and intra-list diversity; p95 recommendation latency; hide/report/complaint rate; and a minimum popularity floor so the discovery row does not collapse into obscure titles.
- **Decision rule:** ship both only if arm C improves the primary outcome over the better single-row arm without breaching a guardrail. Pre-register the metric definitions and the analysis window before launch; do not choose the winner from a post-hoc subgroup.

Offline Recall@10 cannot establish current-session intent, viewing completion, satisfaction, retention or revenue. It also cannot tell us whether a less popular list is a better discovery experience or simply a worse match. The online experiment, not the offline table, must decide deployment.

8. Limitations and disclosures

- The split is chronological within each user, not globally time-consistent. MovieLens records rating times, not viewing sessions.
- Evaluation is warm-user only; users without positive training edges are excluded, so no cold-start claim is made.
- The 13 relevant test IDs absent from the training candidate pool cannot be retrieved by any method and are reported rather than hidden.
- MF and PPR use different validation objectives (RMSE vs Recall@10), so their test recalls are descriptive. The MF L2 correction is disclosed in section 4.2; without it the reference collapsed to the global mean.
- Popularity is counted from training ratings only and is descriptive, not a platform impression count.

- Timings come from one headless Chrome/WebGL run on the stated GPU and are not compared across devices.
- The unmodified AI-session export is an administrative submission step and is not embedded in this report.

9. Reproducibility

From `week6/`, run `py -m http.server 8000`, open `http://localhost:8000/index.html`, then press Build graph, Run tests, Run alpha sweep, Evaluate Recall@10, and Run two-user demo. TensorFlow.js 4.22.0 is loaded from the CDN; D3 7.9.0 is local. The recorded run used Chrome 155.0.8059.85 with the WebGL backend. Exports are `verification.json`, `results.csv` and `recommendations.csv`.

10. Independent verification and author's reflection

Verification statement. I, Ilya Bochkarev, independently launched the submitted application from `week6/` with `py -m http.server 8000` and repeated the documented workflow in Chrome on 22 October 2026.

- Build graph reported 2,554 nodes and 45,602 positive training edges.
- Self-checks reported 12 passed, 0 failed and 0 pending.
- The validation alpha sweep selected alpha 0.50.
- Evaluation completed with the same selected alpha.
- The two-user demonstration completed for users 1 and 2 with one factor changed per comparison.

These observations match the settings and control flow recorded in the checked-in artifacts. The numerical tables in this report come from those exports; the hand calculation in section 4.4 independently checks one propagation step. Alpha was selected on validation, not tuned on the test split.

What this work gave me. The main result is not that PPR is newer or automatically better. It is that a graph method can be compared with two baselines and a previous model under one shared cohort, and that the comparison can support a specific deployment choice. I now understand alpha as an exploration–restart trade-off rather than a quality knob: it has to be selected on validation, and a larger value is not automatically better. I also understand why MF and PPR should not share a score scale: one predicts ratings, while the other ranks graph proximity. Finally, the offline numbers justify a hypothesis, not a rollout; the proposed A/B test is the next step before claiming engagement gains.

11. Execution log and evidence trail

This section records the order in which I ran the submitted application and the evidence I inspected. It is deliberately separate from the results: Section 5 reports what the numbers are, while this section shows how I produced and checked them. The screenshots below were captured during the run; the numeric tables were then cross-checked against the exported CSV and JSON files.

11.1 Step-by-step record

#	What I did	What I observed	Evidence
1	Selected the coding model in OpenCode and started the session.	DeepSeek V4.1 Flash was selected; the first screenshot is timestamped 15:03.	Fig. 1; `session.json`
2	Started the local HTTP server from `week6/` and opened `http://localhost:8000/index.html`.	The page loaded the MovieLens files and reported the TensorFlow.js `webgl` backend; the dataset summary is visible in Fig. 4.	Fig. 4; Section 9
3	Pressed **Build graph**.	943 user nodes, 1,611 movie nodes, 2,554 total nodes, 45,602 positive training edges, 1 zero-degree user and 253 zero-degree movies.	Figs. 2–4
4	Pressed **Run tests**.	The status line reported 12 passed, 0 failed and 0 pending; the individual checks are recorded in `verification.json`.	Fig. 4; `verification.json`
5	Pressed **Run alpha sweep (validation)**.	The sweep completed and the UI reported selected alpha 0.50. The four validation points are 0.1145, 0.1048, 0.0900 and 0.0738 for alpha 0.50, 0.70, 0.85 and 0.95.	Fig. 5; `results.csv`
6	Pressed **Evaluate Recall@10 — all four methods**.	The shared TEST cohort contained 861 users; PPR reached Recall@10 0.1122, above global PageRank 0.0797, popularity 0.0760 and MF 0.0312.	`results.csv`; Section 5
7	Pressed **Run two-user demo (alpha / seed)**.	Changing only the restart user gave 0/10 shared titles; changing only alpha from 0.85 to 0.50 gave 7/10 shared titles.	Section 6; `recommendations.csv`
8	Inspected the PPR and MF Top-10 lists for Users 1 and 2.	The lists contained titles, genres, scores and training popularity; the two methods produced different recommendation intents.	Section 6.2–6.3; `recommendations.csv`
9	Worked one PageRank update by hand on the four-node reference graph.	I wrote the transition matrix, the initial vector, the incoming masses, the restart term, and obtained `p1 = [0.25, 0.14375, 0.56875, 0.0375]`; the values sum to 1.	Section 4.4; Fig. 6
10	Exported and compared the machine-readable artifacts, then recorded the session.	`verification.json`, `results.csv` and `recommendations.csv` matched the values used in the report; the final `session.json` is the audit trail.	Section 9; submission files

Note on alpha 0.55. Two screenshots show the slider at 0.55. That was an intermediate manual position used while inspecting the graph view before the validation sweep; it is not a reported result. The selected value used in the evaluation is 0.50.

11.2 Screenshots from the run

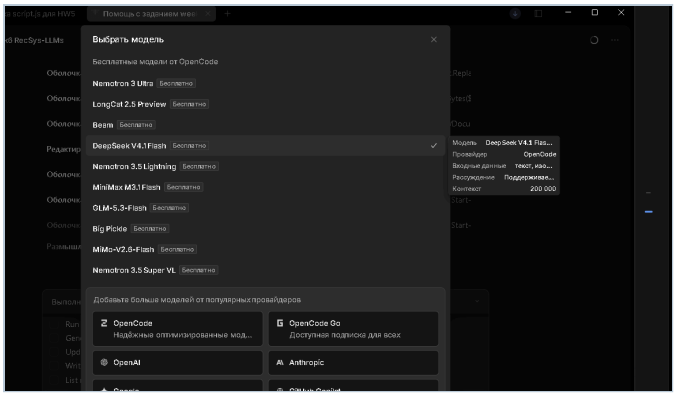


Fig. 1. OpenCode model selection at 15:03. I used DeepSeek V4.1 Flash through OpenCode for the assisted implementation.

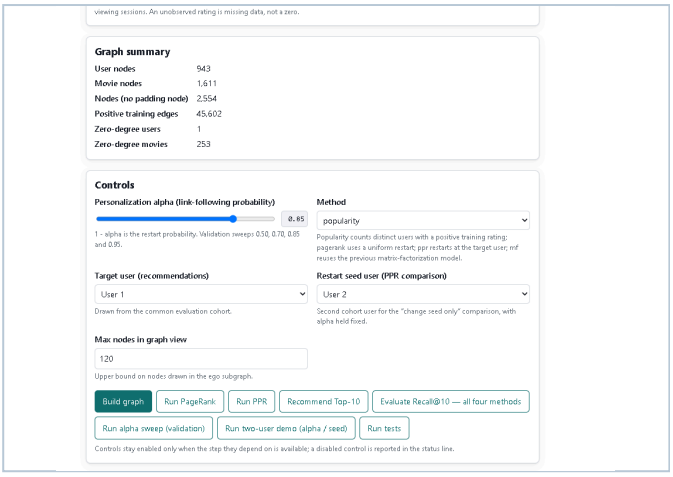


Fig. 2. Graph summary after Build graph: 943 user nodes, 1,611 movie nodes, 2,554 total nodes and 45,602 positive training edges. The controls show User 1 and restart User 2 at alpha 0.85.

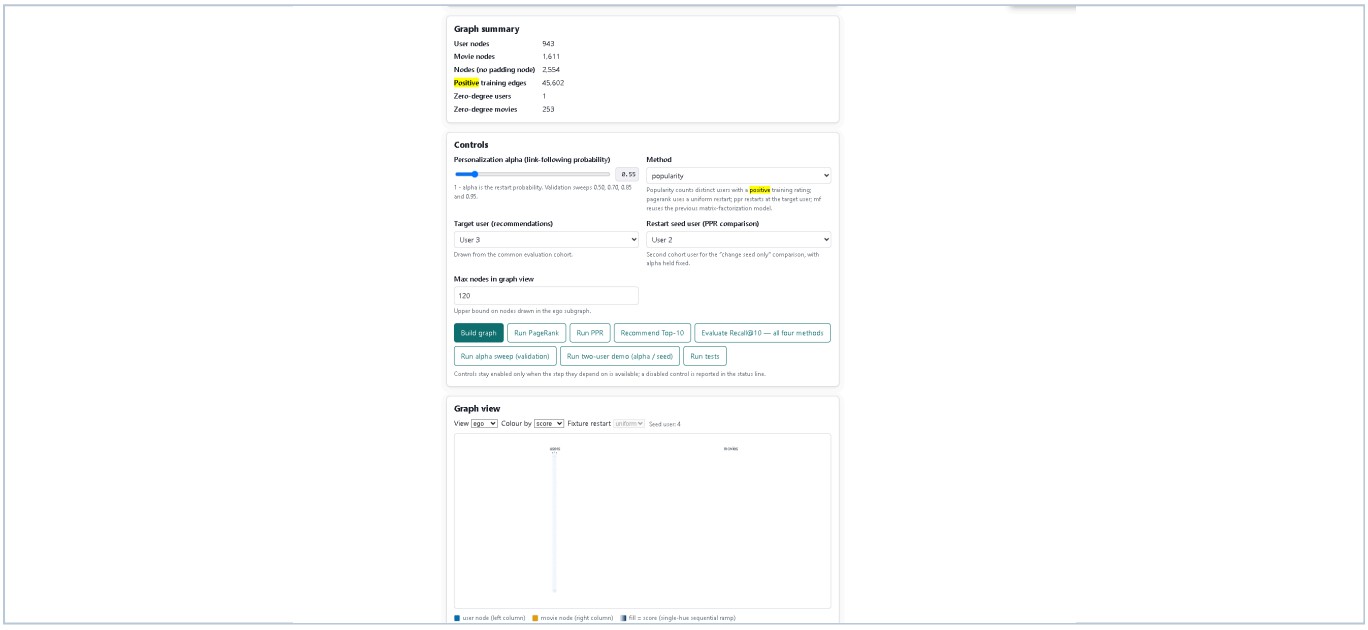


Fig. 3. Graph view at 17:58. The ego subgraph is rendered for the selected seed. Alpha 0.55 is an intermediate manual setting, not the selected value.

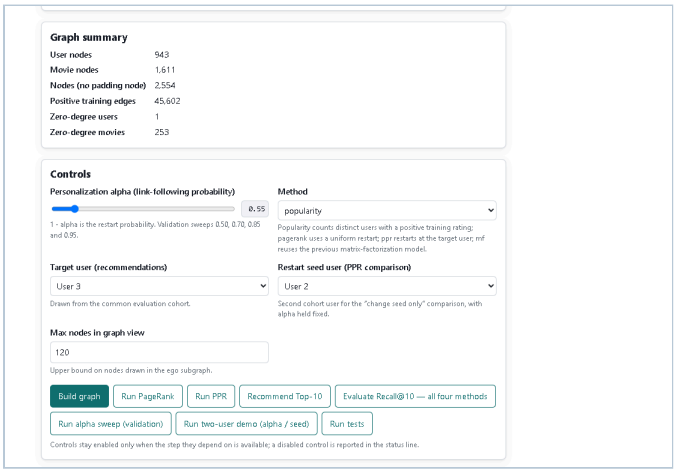


Fig. 4. Dataset and graph summaries at 18:35 with the TensorFlow.js `webgl` backend: 100,000 ratings, split 79,619 / 9,596 / 10,785, and 45,602 positive training interactions. The status line reports 12 passed, 0 failed and 0 pending.

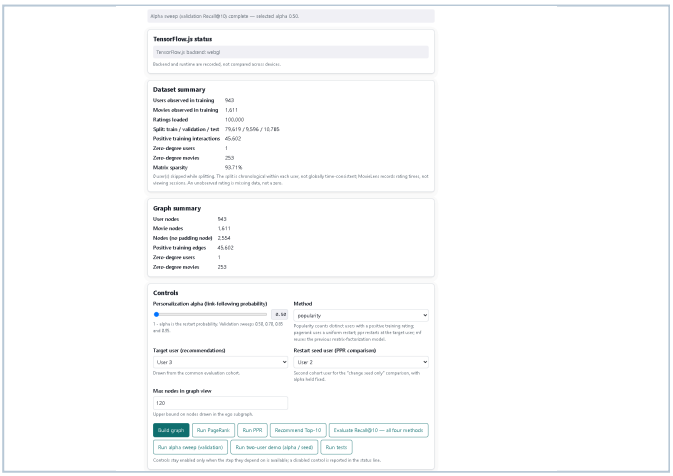


Fig. 5. Validation alpha sweep at 18:35. The status line reports selected alpha 0.50; this is the value used in the four-method evaluation.

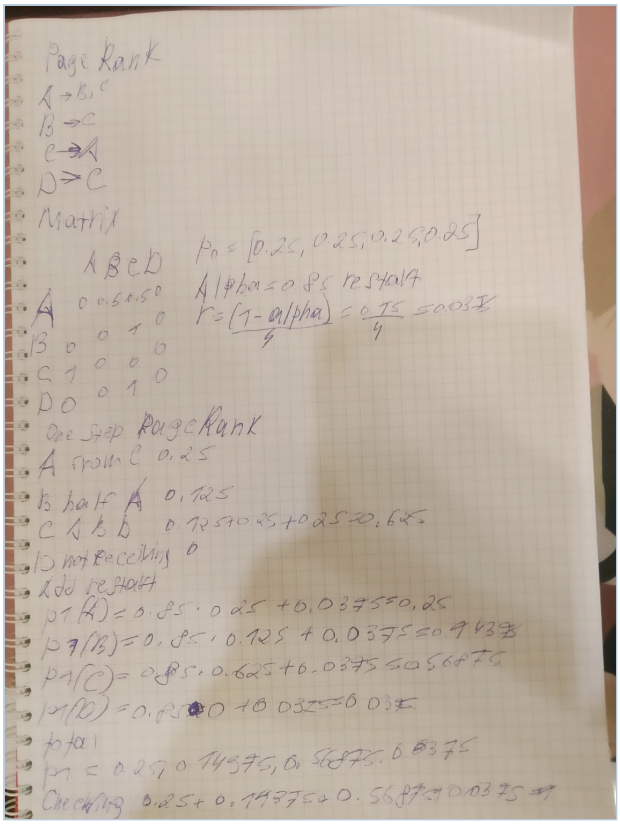


Fig. 6. My handwritten one-step PageRank check on the four-node reference graph. The sheet records the edges, the transition matrix, the initial vector `p0 = [0.25, 0.25, 0.25, 0.25]`, the restart value `r = 0.0375`, the incoming masses and the result `p1 = [0.25, 0.14375, 0.56875, 0.0375]`, followed by the sum check. This verifies one propagation step, not the fixed point.

11.3 What this evidence establishes

The screenshots show the application actually running, the graph being built, the test summary, the validation sweep and the selected alpha. The exported files then connect those UI states to the numerical tables in Sections 5 and 6. The handwritten PageRank calculation in Section 4.4 is the independent arithmetic check; the browser re-run in Section 10 is the independent execution check. Together they show a traceable path from the source code to the reported numbers, rather than only a final result.

References

Brin, S., and Page, L. 1998. The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks and ISDN Systems* 30, 1–7, 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

Harper, F. M., and Konstan, J. A. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4, Article 19. https://doi.org/10.1145/2827872

Jeh, G., and Widom, J. 2003. Scaling Personalized Web Search. *Proceedings of WWW 2003*, 271–279. https://doi.org/10.1145/775152.775191

Koren, Y., Bell, R., and Volinsky, C. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8, 30–37. https://doi.org/10.1109/MC.2009.263

MovieLens 100K dataset. GroupLens Research, University of Minnesota. https://grouplens.org/datasets/movielens/100k/

Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of UAI 2009*, 452–461. https://arxiv.org/abs/1205.2618

TensorFlow.js 4.22.0 API documentation. https://js.tensorflow.org/